

Guía 2: CUDA

Solo Ejercicios de Programación y Rendimiento

Sistemas Distribuidos y Paralelos

Departamento de Ingeniería Informática — Universidad de Santiago de Chile
Prof. Miguel Cárcamo

Semestre 1-2026

Objetivo

Esta guía se enfoca exclusivamente en ejercicios sobre CUDA. Se combinan problemas de modelado teórico (Ley de Amdahl, Ley de Gustafsson, speedup y eficiencia), análisis de código, configuración de grilla/bloques/hebras, transferencias de memoria, y comparación de resultados entre versiones secuenciales, OpenMP y CUDA.

Convenciones para toda la guía

- Speedup: $S = \frac{T_{\text{seq}}}{T_{\text{par}}}$
- Eficiencia: $E = \frac{S}{P}$, donde P es cantidad de workers paralelos (threads CPU o núcleos CUDA efectivos, según el contexto del ejercicio).
- Ley de Amdahl: $S(P) = \frac{1}{s + \frac{1-s}{P}}$
- Ley de Gustafsson (misma convención usada en guías del curso): $S(P, W) = \frac{P}{1 + (P-1)s(W)}$, donde $s(W)$ disminuye cuando crece el tamaño del problema W .

1. Modelado Teórico (Amdahl, Gustafsson, Speedup, Eficiencia)

Ejercicio 1. Amdahl aplicado a offloading CUDA

Un algoritmo tiene:

- 15 % secuencial fijo en CPU (pre/post-procesamiento).
- 85 % potencialmente acelerable en GPU.

Considere $P = \{128, 256, 512, 1024, 2048\}$ unidades efectivas para la parte paralela.

- Calcule $S(P)$ con Amdahl.
- Calcule $E(P)$.
- Determine el límite $S_{\text{máx}}$ cuando $P \rightarrow \infty$.
- Interprete por qué aumentar mucho P no entrega ganancias proporcionales.

Ejercicio 2. Amdahl con overhead de transferencia

Para un kernel de suma de vectores:

$$T_{\text{total,CUDA}}(N) = T_{H2D}(N) + T_{\text{kernel}}(N) + T_{D2H}(N)$$

con:

$$T_{H2D} = 0,2 + 0,0008N, \quad T_{\text{kernel}} = 0,0006N, \quad T_{D2H} = 0,2 + 0,0008N$$

(tiempos en ms), y $T_{\text{seq}}(N) = 0,01N$ ms.

- (a) Calcule el speedup total para $N = 10^4, 10^5, 10^6, 10^7$.
- (b) Calcule speedup considerando solo kernel.
- (c) Estime la fracción secuencial efectiva $s(N) = \frac{T_{H2D} + T_{D2H}}{T_{\text{total,CUDA}}}$.
- (d) Discuta si Amdahl o Gustafsson describe mejor el comportamiento al crecer N .

Ejercicio 3. Ley de Gustafsson con tamaño de problema creciente

Un algoritmo en GPU escala el tamaño de entrada con P , manteniendo tiempo total cercano a constante. Se mide $s(W) = 0,06$ para $P = 256$, $s(W) = 0,04$ para $P = 512$, $s(W) = 0,025$ para $P = 1024$.

- (a) Calcule $S(P, W) = \frac{P}{1 + (P-1)s(W)}$ para cada caso.
- (b) Compare con Amdahl usando $s = 0,06$ fijo para todos los P .
- (c) Explique el impacto de disminuir la fracción secuencial al crecer el problema.
- (d) Concluya cuándo usar Amdahl y cuándo Gustafsson en CUDA.

Ejercicio 4. Eficiencia y punto de saturación

Para un kernel se obtienen tiempos medidos:

$$T_{\text{seq}} = 120 \text{ ms}, \quad T_{\text{CUDA}} = \{22, 14, 11, 10, 9, 8\} \text{ ms}$$

asociados a configuraciones de ejecución equivalentes a $P = \{256, 512, 1024, 1536, 2048\}$.

- (a) Calcule speedup y eficiencia para cada P .
- (b) Determine desde qué configuración hay rendimientos marginales decrecientes fuertes.
- (c) Proponga una explicación arquitectural (ocupación, ancho de banda, latencia, etc.).

2. Kernel, Modificadores y Configuración de Ejecución

Ejercicio 5. Diferencias __host__, __device__, __global__

Analice el siguiente código:

```
__host__ float f_cpu(float x) { return x + 1.0f; }
__device__ float f_gpu(float x) { return x * x; }
__global__ void k(float *in, float *out, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) out[i] = f_gpu(in[i]);
}
```

- (a) Explique dónde se ejecuta cada función.
- (b) Indique cuáles llamadas son válidas y cuáles no:

- desde main a f_cpu
- desde main a f_gpu
- desde k a f_gpu
- desde k a f_cpu

(c) ¿Por qué `__global__` retorna void?

Ejercicio 6. Cálculo de grilla y bloques 1D/2D/3D

- Para $N = 10^6$, calcule `grid.x` usando `block.x=256`.
- Para una imagen $H \times W = 1080 \times 1920$, proponga `block=(16,16)` y calcule `grid=(?,?)`.
- Para un volumen $D \times H \times W = 64 \times 256 \times 256$ con `block=(8,8,4)`, calcule `grid=(?,?,?)`.
- Escriba las expresiones de índice global para 2D y 3D.

Ejercicio 7. Verificación de índices y límites

Analice el kernel:

```
__global__ void blur2d(const float *in, float *out, int H, int W) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int idx = row * W + col;
    out[idx] = 0.25f * (in[idx] + in[idx-1] + in[idx+1] + in[idx+W]);
}
```

- Identifique al menos cuatro errores/riesgos en términos de límites de memoria.
- Proponga una versión segura con validaciones de borde.
- Explique cómo estos errores afectan rendimiento y correctitud.

Ejercicio 8. Selección de blockDim: criterio práctico

Para tres kernels distintos se tiene esta información:

- K1: bound por memoria global, pocos registros/thread.
- K2: alto uso de registros/thread, poco acceso a memoria.
- K3: uso intenso de shared memory por bloque.

- Proponga un rango de `blockDim` inicial para cada kernel.
- Explique cómo influyen registros y shared memory en bloques residentes por SM.
- Relacione el tamaño de bloque con occupancy y throughput real.

3. Transferencias y Gestión de Memoria CUDA

Ejercicio 9. Tipos de transferencia cudaMemcpy

Complete y explique:

- Transferencia Host a Device: _____
- Transferencia Device a Host: _____
- Transferencia Device a Device: _____

Luego responda:

- (d) ¿En qué caso usaría `DeviceToDevice` en una aplicación real?
- (e) ¿Qué errores comunes ocurren al invertir origen/destino?

Ejercicio 10. Modelo de costo total con múltiples kernels

Una pipeline ejecuta 3 kernels consecutivos sobre los mismos datos:

$$H2D \rightarrow K_1 \rightarrow K_2 \rightarrow K_3 \rightarrow D2H$$

con:

$$T_{H2D} = 3,2 \text{ ms}, T_{K_1} = 1,0 \text{ ms}, T_{K_2} = 2,5 \text{ ms}, T_{K_3} = 0,9 \text{ ms}, T_{D2H} = 3,1 \text{ ms}$$

y versión secuencial CPU de 21 ms.

- (a) Calcule speedup total.
- (b) Calcule speedup de cómputo puro GPU ($K_1 + K_2 + K_3$).
- (c) Estime porcentaje del tiempo total dedicado a transferencias.
- (d) Discuta una estrategia para mejorar speedup sin cambiar kernels.

4. Comparación Secuencial vs OpenMP vs CUDA

Ejercicio 11. SAXPY: comparación de tres versiones

Considere:

```
// Secuencial
for (int i = 0; i < N; ++i) y[i] = a * x[i] + y[i];

// OpenMP
#pragma omp parallel for
for (int i = 0; i < N; ++i) y[i] = a * x[i] + y[i];

// CUDA (kernel + copias)
saxpy<<<grid,block>>>(a, d_x, d_y, N);
```

Mediciones para $N = 2 \times 10^7$:

$$T_{\text{seq}} = 140 \text{ ms}, T_{\text{OpenMP}} = 34 \text{ ms}, T_{\text{CUDA, total}} = 19 \text{ ms}, T_{\text{CUDA, kernel}} = 7 \text{ ms}$$

- (a) Calcule speedup de OpenMP y CUDA total respecto a secuencial.
- (b) Calcule speedup “kernel-only” de CUDA.
- (c) ¿Qué porcentaje del tiempo CUDA total corresponde a transferencias+overhead?
- (d) ¿Qué implementación elegiría para $N = 10^5$? Justifique con modelo cualitativo.

Ejercicio 12. Histograma: impacto de atomics

Tiempos medidos para un histograma con 256 bins:

- Secuencial: 220 ms
- OpenMP con `critical`: 180 ms
- OpenMP con histograma privado + merge: 58 ms
- CUDA naive con `atomicAdd` global: 41 ms
- CUDA optimizado (shared + merge): 19 ms

- (a) Calcule speedup de cada versión respecto a secuencial.
- (b) Ordene las implementaciones por eficiencia práctica.
- (c) Explique por qué la versión CUDA optimizada supera a la naive.
- (d) Relacione estos resultados con fracción serial efectiva y contención.

Ejercicio 13. Matriz-matriz: análisis por tamaño

Tiempos (ms):

Tamaño $N \times N$	Secuencial	OpenMP	CUDA total
256×256	44	15	20
1024×1024	2700	520	180
2048×2048	21600	4200	890

- (a) Calcule speedup para OpenMP y CUDA en cada tamaño.
- (b) Identifique en qué tamaño CUDA pasa a ser claramente dominante.
- (c) Discuta este resultado en términos de overhead fijo y escalabilidad.
- (d) ¿Qué ley explica mejor la tendencia al crecer N ?

5. Depuración y Perfilado en CUDA

Ejercicio 14. cuda-gdb vs gdb: uso práctico

- (a) Indique tres similitudes operativas entre `gdb` y `cuda-gdb`.
- (b) Indique tres diferencias clave para depurar kernels.
- (c) Explique por qué compilar con `-G` ayuda a depuración pero puede distorsionar tiempos.
- (d) Proponga un flujo mínimo de depuración para un bug intermitente en kernel.

Ejercicio 15. Lectura de métricas de Nsight

Para un kernel se reporta:

- Achieved Occupancy: 37 %
- DRAM Throughput: 82 % del pico
- Warp Execution Efficiency: 61 %
- SM Issue Active: 44 %

- (a) Determine si el kernel parece bound por memoria o por cómputo.
- (b) Proponga dos hipótesis de optimización prioritarias.
- (c) Indique qué métrica esperaría mejorar tras cada hipótesis.
- (d) Diseñe un mini plan de validación experimental (antes/después).

6. Ejercicios Integradores CUDA

Ejercicio 16. Diseño de benchmark: secuencial vs OpenMP vs CUDA

Diseñe un protocolo experimental para comparar tres implementaciones de un mismo problema (por ejemplo, convolución 1D o SAXPY).

- (a) Defina tamaños de entrada y número de repeticiones.
- (b) Defina qué tiempos medirá por separado (CPU total, GPU kernel, H2D, D2H, end-to-end).
- (c) Defina métricas: speedup, eficiencia, throughput.
- (d) Defina criterios de correctitud numérica.
- (e) Defina cómo reportará resultados (tabla/gráfico) y conclusiones esperadas.

Ejercicio 17. Amdahl + Gustafsson con datos experimentales

Se midieron speedups CUDA end-to-end:

$$S = \{1, 8, 3, 2, 5, 9, 8, 7\}$$

para tamaños crecientes $\{N_1, N_2, N_3, N_4\}$, manteniendo la misma GPU.

- (a) Estime una fracción secuencial efectiva s usando Amdahl para cada tamaño.
- (b) Analice la tendencia de s al crecer N .
- (c) Interprete si hay evidencia de escalabilidad tipo Gustafsson.
- (d) Proponga dos cambios concretos para aumentar aún más el speedup end-to-end.